

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 6

Erros, Logging e Depuração

Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025

CAPÍTULO 6

Erros, Logging e Depuração

Como debugar além do `print()`

Engenharia de Software para Ciência de Dados

Roteiro da Aula

> **Parte 1: Entendendo Erros**

- > O problema do `print()`
- > Tipos de Erros (Sintaxe vs Runtime)
- > Lendo o Stack Trace
- > Tratamento de Exceções ('try-except')

> **Parte 2: Logging**

- > Por que usar Logs?
- > Níveis de Log (INFO, ERROR, DEBUG)

> **Parte 3: Depuração (Debugging)**

- > O Método Científico
- > Debuggers Interativos (PDB/IDE)

O Vício do print()

Muitos cientistas de dados dependem exclusivamente de declarações `print()` para encontrar bugs.

Por que isso é ruim para produção?

- > Polui a saída padrão (stdout).
- > Difícil de desligar/ligar globalmente.
- > Não persiste informações (se o console fechar, adeus histórico).
- > Falta contexto (timestamp, arquivo, linha).



Classificando Erros



Erros de Sintaxe

Ocorrem **antes** do código rodar. O Python não entende o que você escreveu.

`SyntaxError: invalid syntax`



Erros de Tempo de Execução

Ocorrem **durante** a execução. O código é válido, mas algo deu errado (divisão por zero, arquivo não encontrado).

`ZeroDivisionError, KeyError`



Erros Lógicos

O código roda sem travar, mas produz o **resultado errado**. São os mais difíceis de encontrar.

Ex: Média calculada errada.

O que é um Stack Trace?

O Stack Trace (Rastreamento de Pilha) é o mapa do crime. Ele mostra a sequência de chamadas de função que levou ao erro.

```
Traceback (most recent call last): File "analise.py", line 42, in  
  processar_dados() File "analise.py", line 15, in processar_dados resultado = 100 / divisor  
ZeroDivisionError: division by zero
```

Dica de Ouro: Leia de baixo para cima! A última linha diz *o que* aconteceu, as linhas acima dizem *onde*.

Lendo o Rastro

Não entre em pânico com a parede de texto vermelho.

- > Identifique o **Tipo da Exceção** no final.
- > Encontre a **linha do seu código** que disparou o erro (ignore bibliotecas internas por enquanto).
- > Verifique os valores das variáveis naquele ponto.

ed.ipynb

le

```
s://raw.githubusercontent.com/justmarkham/DAT8/master/dat  
)sum()['item_price']
```

```
-----  
Traceback (most recent call last)  
lib\site-packages\pandas\core\indexes\base.py in get_loc(  
return self._engine.get_loc(key)  
KeyError:  
  
pandas._libs.index.IndexEngine.get_loc()  
  
pandas._libs.index.IndexEngine.get_loc()  
  
ass_helper.pxi in pandas._libs.hashtable.PyObjectHashTabl  
ass_helper.pxi in pandas._libs.hashtable.PyObjectHashTabl  
  
ove exception, another exception occurred:
```

Tratamento de Exceções (`try-except`)

Permite que seu programa lide com erros graciosamente em vez de travar (crash).

```
try: arquivo = open("dados.csv") dados = arquivo
```

Boas Práticas

Envolva apenas o código que pode falhar no bloco try.

Sempre capture exceções **específicas**.

Evite `except Exception`

✗ O jeito errado

```
try: codigo_complexo() except Exception: pass # Engole TODOS os erros!
```

Isso esconde bugs reais (como erros de digitação) e torna a depuração impossível.

✓ O jeito certo

```
try: codigo_complexo() except
```

Captura apenas o que você espera e

Blocos `else` e `finally`

- > `else`: Roda se **nenhuma** exceção ocorrer no `try`.
- > `finally`: Roda **sempre**, havendo erro ou não. Crucial para limpeza de recursos.

```
try: conexao = db.connect() except ConnectionError: log_erro() else: print("Conexão bem sucedida!") finally:
```

Falhe Rápido (`raise`)

Às vezes, é melhor parar o programa do que continuar com dados inválidos.

```
def calcular_raiz(x): if x < 0: raise ValueError("x não pode ser negativo") return math.sqrt(x)
```

Usar raise força quem chama sua função a lidar com o problema ou corrigir os dados de entrada.

Exceções Personalizadas

Crie exceções específicas para o seu domínio de problema para tornar o código mais legível.

```
class ModeloNaoTreinadoError(Exception): """Levantada ao tentar prever sem treinar.""" pass # No código
```



Parte 2: Logging

O diário de bordo do seu software

O que é Logging?

Logging é a prática de registrar eventos que acontecem durante a execução do software.

Diferente do print, o logging permite:

- > Categorizar a importância da mensagem.
- > Escrever em arquivos, bancos de dados ou serviços de nuvem.
- > Filtrar mensagens sem alterar o código.

```
h m i tse s o .u c
a s eOo t s-hd' p eh id l
c t h h i rtne vt wn e
r t f ea Al ' a a v
y a e htr t s rO t a
a p h h t tset p b se r
s m g t i ' a e ed b
a u neO b t e la b
O r o d ,s ih s h Bm e
r n pA t e t t h h
e h a e s ho h t
h t e es to f t
t e t ni f o a f
f e s iT d h o
r o r h' n l e
e o f g s au m . t e
' r n o o n a m
O p e i w r f h o h o
h r o a i t h
e t e n w r e t
v , ,w i h a r e
, a fpso d fe t le h
G e e t e h e u t
```

Níveis de Log (Hierarquia)

DEBUG

Detalhes técnicos para diagnóstico.

INFO

Confirmação de que tudo vai bem.

WARNING

Algo inesperado, mas ainda funciona.

ERROR

Falha em uma função específica.

CRITICAL

O programa inteiro pode parar.

Se você configurar o nível para **WARNING**, mensagens de **DEBUG** e **INFO** serão ignoradas.

O Módulo `logging`

Biblioteca padrão do Python, robusta e flexível.

```
import logging logging.basicConfig(level=logging.INFO) logger = logging.getLogger(__name__) logger.info
```

Handlers e Formatters

Handlers (Para onde vai?)

- > StreamHandler: Console (stdout/stderr).
- > FileHandler: Arquivo de texto (.log).
- > SMTPHandler: Envia por email.

Formatters (Como se parece?)

Define o layout da mensagem.

```
'%(asctime)s - %(name)s - %(levelname)s - %(message)s'
```

2023-10-27 10:30:00 - app.core - ERROR - Falha na conexão

Logging em Data Pipelines

Para pipelines longos (ETL, treinamento de modelos), logs são vitais.

- > **Início/Fim:** "Iniciando treinamento da época 1/100".
- > **Métricas:** "Acurácia atual: 0.85".
- > **Formato dos Dados:** "DataFrame shape: (1000, 50)".

Isso permite monitorar o progresso sem precisar de acesso interativo.



Parte 3: Depuração (Debugging)

Encontrando a agulha no palheiro

O Método Científico de Debugging

1

Observe

Reproduza o erro consistentemente. Entenda o que está acontecendo vs. o que deveria acontecer.

2

Hipótese

Formule uma teoria. "Acho que a variável X está vula porque o arquivo não carregou."

3

Teste

Use o debugger ou logs para confirmar a hipótese. Não mude código aleatoriamente!

Debuggers Interativos (PDB/IPDB)

O Python vem com um debugger embutido ('pdb'). O 'ipdb' é uma versão mais amigável (cores, autocomplete).

Permite **pausar** a execução e inspecionar o estado.

```
import ipdb def funcao_complexa(x): y = x * 2 ipdb.set_trace() # 0 programa PAUSA aqui! return y + 10
```

Comandos Essenciais do Debugger

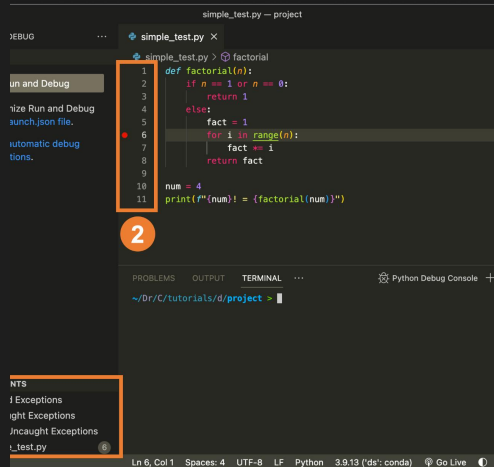
Comando	Ação
n (next)	Executa a próxima linha (pula chamadas de função).
s (step)	Entra dentro da chamada de função atual.
c (continue)	Continua a execução até o próximo breakpoint.
p variavel	Imprime o valor de uma variável.
q (quit)	Sai do debugger e aborta o programa.

Debugging Visual (VS Code / PyCharm)

IDEs modernos oferecem interfaces gráficas poderosas para depuração.

- > Veja todas as variáveis locais e globais em um painel lateral.
- > Pilha de chamadas (Call Stack) visual.
- > Console interativo no contexto da pausa.

Muitas vezes mais rápido e intuitivo que comandos de terminal.



Breakpoints e Watchpoints

Breakpoints (Pontos de Parada)

Marcadores vermelhos ao lado do número da linha.
Dizem ao IDE: "Pare aqui antes de executar esta linha".

Conditional Breakpoints

Uma ferramenta poderosa: "Pare aqui, MAS APENAS SE `i == 500`".

Crucial para debugar loops longos onde o erro só acontece na iteração 500.

Resumo das Melhores Práticas

- > **Falhe Graciosamente:** Use try-except para erros esperados.
- > **Falhe Rápido:** Use raise se os dados estiverem errados.
- > **Log, não Print:** Use logging para sistemas em produção.
- > **Pare, não Adivinhe:** Use um debugger para inspecionar o estado real.



CLEAN CODE
PRINCIPLES:
BE A BETTER
PROGRAMMER



Dúvidas?

Obrigado!